

HIGH-ALTITUDE VEHICLE DETECTION FULL PIPELINE REPORT

VisDrone2019-DET | Vehicle Filtering | Blur+Noise Degradation | Real-ESRGAN SR (2x/4x/8x) | SAHI Slicing | RT-DETR | Loss Fine-Tuning

1. PROJECT OVERVIEW

Component	Detail
Dataset	VisDrone2019-DET — Full dataset (6,471 train + 548 val)
Task	High-altitude vehicle detection from drone at ~6 km altitude
Target classes	8 vehicle classes: bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor
Non-vehicle skip	Pedestrian (ID 1) and People (ID 2) excluded — not vehicles
Compute	Kaggle T4 GPU (30 hrs/week) + Google Colab T4 GPU
Storage	Google Drive — persistent storage via rclone sync
SR method	Real-ESRGAN (switched from SwinIR — 4x faster, comparable quality)
SR scales	2x, 4x, 8x (4x chain + 2x pass)
Detection model	RT-DETR-L — Real-Time Detection Transformer Large
Inference method	SAHI — Slicing Aided Hyper Inference (640x640 tiles, 20% overlap)
Loss fine-tuning	Pixel loss (L1) + Perceptual loss (VGG19) + Adversarial loss (PatchGAN)

2. COMPLETE PIPELINE ARCHITECTURE

The project was executed across two platforms with Google Drive as the persistent storage bridge. The pipeline is structured in three phases: data preparation on Colab, super-resolution and training on Kaggle, and evaluation across all zoom and degradation conditions.

Stage	Operation	Platform	Output
1	Load full VisDrone2019-DET dataset	Kaggle / Colab	6,471 train + 548 val images
2	Filter vehicle-only images (8 classes)	Colab	Images containing at least one vehicle, YOLO labels
3	Rotational augmentation (90/180/270 + random)	Colab	~6x dataset expansion via symlink-free manifest approach
4	Gaussian blur + noise degradation	Colab	Degraded dataset: 11,098 train + 3,276 val
5	Save degraded dataset to Google Drive	Colab	Persistent backup — survives session disconnects
6	Fix image-label mismatch in degraded dataset	Kaggle	Clean dataset with equal image and label counts
7	Real-ESRGAN 2x super-resolution on degraded	Kaggle	SR 2x dataset synced to Drive every 500 images
8	Real-ESRGAN 4x super-resolution on	Kaggle	SR 4x dataset synced to Drive

	degraded		
9	Real-ESRGAN 8x (4x then 2x chain) on degraded	Kaggle	SR 8x dataset synced to Drive
10	SAHI slicing into 640x640 tiles (all 3 zoom levels)	Kaggle	Sliced training tiles for each zoom level
11	Write YAML configs for 2x / 4x / 8x datasets	Kaggle	Three dataset.yaml files
12	Train RT-DETR on 2x sliced tiles	Kaggle	Trained model + mAP@50 results
13	Train RT-DETR on 4x sliced tiles	Kaggle	Trained model + mAP@50 results
14	Train RT-DETR on 8x sliced tiles (standard)	Kaggle	Standard 8x trained model
15	8x frozen backbone evaluation	Kaggle	Frozen encoder, head-only mAP
16	8x joint SR + detection training	Kaggle	End-to-end jointly trained model
17	Loss fine-tuning (pixel + perceptual + adversarial)	Kaggle	Fine-tuned best model — improved mAP
18	Final comparison plots across all conditions	Kaggle	Bar charts, line curves, radar chart

3. PHASE 1 — DATA PREPARATION (COLAB)

3.1 Vehicle Filtering

The complete VisDrone2019-DET dataset was loaded and filtered to keep only images containing at least one valid vehicle annotation. Unlike a tiny-object pipeline which would also filter by bounding box area, this pipeline retains vehicles of all sizes — from large buses occupying hundreds of pixels to small motorcycles barely visible at altitude. The goal is robust vehicle detection across the full size range encountered at 6 km altitude.

Category ID	Class Name	Action	YOLO Re-index
3	Bicycle	KEEP	0
4	Car	KEEP	1
5	Van	KEEP	2
6	Truck	KEEP	3
7	Tricycle	KEEP	4
8	Awning-tricycle	KEEP	5
9	Bus	KEEP	6
10	Motor	KEEP	7
0	Ignored region	SKIP	—
1	Pedestrian	SKIP	—
2	People	SKIP	—
11	Others	SKIP	—

The VisDrone annotation format (x, y, w, h, score, category_id, truncation, occlusion) was converted to YOLO normalised format (class cx cy w h) during the filter step. Only annotations with score=1 (valid, non-ignored) were converted. Bounding box coordinates were clamped to image bounds before normalisation to prevent values outside [0, 1].

3.2 Rotational Data Augmentation

Rotational augmentation was applied to every filtered vehicle image to improve model robustness to drone heading angle. In real ISR operations the drone may approach a target from any direction, so the model must recognise vehicles regardless of their orientation in the image frame.

Augmentation Type	Angles / Range	Copies per Image	Purpose
Fixed rotation	90 degrees	1	Perpendicular approach — vehicle appears sideways
Fixed rotation	180 degrees	1	Opposite heading — vehicle appears flipped
Fixed rotation	270 degrees	1	Perpendicular reverse — vehicle appears sideways other way
Random rotation	-30 to +30 degrees	2	Oblique approach angles simulation
Original	0 degrees	1	Baseline — no rotation

The bounding box rotation algorithm transforms all four corners of each box using a 2D rotation matrix centred on the image centre. The axis-aligned bounding box of the four rotated corners is then computed. For non-90-degree

rotations the canvas is expanded to prevent clipping. Box coordinates are then rescaled to the new canvas dimensions and clamped to [0.001, 0.999].

! Problem — Google Drive FUSE Does Not Support Symlinks

The augmentation cell was designed to create symlinks for original images to save ~13GB of disk space (only rotated versions would occupy real storage). When running on Google Colab with Drive mounted, all `os.symlink` and `Path.symlink_to` calls failed silently. The FUSE driver used by Google Drive does not implement the symlink system call.

✓ Fix — Manifest JSON Instead of Copying Originals

Originals were never copied to the augmented directory at all. Instead a `manifest.json` was written mapping each image stem to its path in the FILTERED directory. Downstream cells read originals directly from FILTERED using the manifest. Only the 5 rotated versions per image were saved as actual files, saving approximately 13GB of Drive space.

3.3 Blur + Noise Degradation

All augmented vehicle images were degraded with two sequential operations to simulate real atmospheric and sensor conditions at 6 km altitude. Gaussian blur is applied first to simulate atmospheric haze and optical defocus. Gaussian noise is added second to simulate thermal sensor noise and electronic interference in the drone's imaging payload.

Step	Operation	Parameters	Physical Cause Simulated
1	Gaussian blur	Kernel (5,5), sigma auto-computed	Atmospheric haze, turbulence, lens defocus
2	Gaussian noise	Mean 0, standard deviation 25	Sensor thermal noise, ADC quantisation error

Labels are completely unchanged by both degradation steps. A vehicle at normalised coordinates ($cx=0.5$, $cy=0.3$) occupies exactly the same pixels after blurring and noise — only pixel intensity values change, not spatial object positions. Labels created during the filtering step are valid for all downstream degraded and SR variants without modification.

The degraded output contained 11,098 training images and 3,276 validation images. These numbers reflect the full augmented dataset (original + 5 rotated versions per filtered image) after degradation.

4. DATASET PROBLEMS ENCOUNTERED ON KAGGLE

4.1 Image-Label Mismatch After Upload

After uploading the degraded dataset to Kaggle as a dataset, a serious mismatch was discovered between image and label file counts. The degraded dataset had 812 images but only 501 label files in train, and 751 images with 463 labels in val. Separately, the degraded-train dataset had 1,184 images but zero label files. The SR 4x dataset was critically incomplete with only 202 train and 53 val images.

Kaggle Dataset	Split	Images	Labels	Problem
degraded	train	812	501	311 labels missing
degraded	val	751	463	288 labels missing
degraded-train	train	1,184	0	Labels folder missing entirely
degraded-val	val	997	847	Most complete — used as val source
real-esrgan sr2x	train	552	421	Incomplete — session timed out
real-esrgan sr2x	val	423	276	Incomplete — session timed out
real-esrgan sr4x	train	202	55	Critically incomplete
real-esrgan sr4x	val	53	3	Critically incomplete

! Problem — Session Timeouts Left SR Datasets Incomplete With Missing Labels

Kaggle GPU sessions time out after approximately 9 hours. The ESRGAN processing cells ran out of compute time mid-way through the dataset. Images were written incrementally but the label copy step (which follows the image loop) was never reached before the session ended. Resuming in a new session re-ran the image loop (correctly skipping already-done images) but the label files from the incomplete first run were never created.

✓ Fix — Label Repair Cell + Resume-Safe Processing

A dedicated label repair cell was written that scanned each SR directory, matched images to labels from the degraded source by filename stem, and created empty label files for any image with no corresponding annotation. Empty label files are valid YOLO training samples representing background images with no objects — they help reduce false positives. All processing cells were also updated with disk space guards that stop cleanly at 200MB remaining rather than crashing mid-write.

4.2 rclone Google Drive Connection Problems

! Problem 1 — rclone Binary Not Found After Install Script

The standard rclone install script (`curl https://rclone.org/install.sh | bash`) ran successfully and printed rclone v1.74.2, but the binary was not in PATH in the Kaggle environment. Running `rclone --version` immediately after returned command not found.

✓ Fix 1 — Direct Binary Download to /usr/local/bin

rclone was downloaded as a precompiled zip from the official releases page, extracted, and the binary was copied manually to `/usr/local/bin/rclone` with execute permissions set via `chmod +x`. This bypassed the install script entirely and worked reliably across all subsequent sessions.

! Problem 2 — Invalid JSON Token in rclone Config

After running `rclone authorize` on the local PC, the output token was pasted into the Kaggle config

cell. The connection failed with: drive: failed when making oauth client: invalid character e looking for beginning of value. The pasted value was a base64 string, not the JSON object rclone expects in its config file.

✓ **Fix 2 — Run rclone config Interactively to Generate Proper Config**

rclone config was run interactively on the local PC to completion (choosing Google Drive, leaving client ID blank, scope=drive, auto config = No, shared drive = No, keep remote = Yes, quit). This wrote a properly formatted ~/.config/rclone/rclone.conf file containing the full JSON token. That file's contents were copied directly into the Kaggle config cell.

! **Problem 3 — rclone.conf File Not Found on Local PC**

After running rclone config, cat ~/.config/rclone/rclone.conf returned: No such file or directory. The interactive session had been exited prematurely before the final confirmation step, so the config was never written.

✓ **Fix 3 — Completed Full Interactive Session Including Final Confirmation**

rclone config was run again to full completion. The critical step was answering Yes to keep this remote and then pressing q to quit. Only after the quit step does rclone write the config file. The file was confirmed present with ls -la ~/.config/rclone/ before copying the token.

4.3 NameError on Every Session Restart

! **Problem — NameError: name DEVICE / BASE / LOCAL_DEG is not defined**

Every time the Kaggle kernel restarted (due to timeout, manual restart, or GPU reallocation between sessions), all previously defined Python variables were lost. Running any processing cell would immediately crash with NameError because DEVICE, BASE, SR2_DIR, ESRGAN weights paths, and helper functions were defined in earlier cells that had not been re-executed in the new session.

✓ **Fix — Every Processing Cell Made Fully Self-Contained**

All path variables (WORK_DIR, BASE, LOCAL_DEG, LOCAL_SR2/4/8, WEIGHTS_DIR), device detection (DEVICE, device_str, device_obj), weight paths, and helper function definitions (load_esrgan, esrgan_enhance, upload_to_drive) were duplicated at the top of every processing cell. Each cell now runs independently from a cold kernel start without requiring any other cell to have been executed first.

5. REAL-ESRGAN SUPER-RESOLUTION

5.1 Why Real-ESRGAN Over SwinIR

The original plan specified SwinIR (Swin Transformer-based super-resolution) as the SR backbone due to its state-of-the-art benchmark scores. After implementing and testing SwinIR on a sample batch on Kaggle, the processing time was found to be 8-12 seconds per image. With approximately 14,374 augmented images (11,098 train + 3,276 val) and three zoom levels required, the total compute would have exceeded 100 hours — far beyond Kaggle's 30 hours per week free limit.

Method	Quality (PSNR)	Speed per Image	14K images x3 zooms	Decision
SwinIR-M	Best — SOTA	8-12 seconds	~120 hours	Too slow — rejected
HAT	Best — SOTA+	15-20 seconds	~200 hours	Impractical
Real-ESRGAN	Very good	2-3 seconds	~30 hours	Selected
BSRGAN	Very good	2-3 seconds	~30 hours	Alternative
FSRCNN	Acceptable	0.1 seconds	~1.5 hours	Quality too low
Bicubic	Poor	Instant	Minutes	Baseline reference only

Real-ESRGAN was the most appropriate choice for this specific degradation pipeline because it was trained on images degraded with exactly the combination used here: Gaussian blur, Gaussian noise, and JPEG compression. The model has learned to invert these specific degradation types, making it physically motivated for this project rather than just a general-purpose SR model.

5.2 ESRGAN Import Error — torchvision Breaking Change

! Problem — ModuleNotFoundError: torchvision.transforms.functional_tensor

Attempting to import RealESRGANer crashed immediately with ModuleNotFoundError: No module named torchvision.transforms.functional_tensor. This error occurred inside basicsr/data/degradations.py which is loaded at import time. The rgb_to_grayscale function was moved from torchvision.transforms.functional_tensor to torchvision.transforms.functional in torchvision 0.16, but the basicsr package had not been updated to reflect this change.

✓ Fix — Direct Source File Patch Using pathlib

The installed basicsr/data/degradations.py file was edited in-place using Python's pathlib: read the file contents, replace the broken import line with the correct one, write back. This patch must be applied once per Kaggle session before any ESRGAN import. The function rgb_to_grayscale is identical in both locations — only the import path changed.

5.3 Disk Space Error During Weight Download

! Problem — OSError: [Errno 28] No space left on device

When downloading the RealESRGAN_x4plus.pth weights (64 MB) to /kaggle/working, the download failed mid-transfer with OSError: [Errno 28] No space left on device. The /kaggle/working directory was full from accumulated SR output images from previous processing runs in the same session.

✓ Fix — Download Weights to /tmp and Add Disk Space Guard

Weights were redirected to /tmp/RealESRGAN_x4plus.pth. The /tmp partition is separate from /kaggle/working and typically has 8GB free regardless of working directory usage. A disk space monitoring function was also added that checks free space before each image and stops processing cleanly if less than 200MB remains, preventing corrupt partial writes.

5.4 SR Scales and Chaining Strategy

Three upscaling factors were applied to create three parallel datasets for comparison. The 2x and 4x models use dedicated pretrained Real-ESRGAN weights for their respective scales. The 8x scale is achieved by chaining two passes: first 4x, then 2x on the output. This two-pass approach is superior to three consecutive 2x passes because the 4x model reconstructs large-scale semantic structure in a single pass while the 2x model then refines fine-grained detail.

Scale	Method	Input	Output	Notes
2x	Single pass — RealESRGAN_x2plus	960x540	1920x1080	Direct 2x learned SR
4x	Single pass — RealESRGAN_x4plus	960x540	3840x2160	Direct 4x learned SR
8x	4x pass then 2x pass on result	960x540	7680x4320	Chained — no native 8x model

After ESRGAN enhancement, labels do not need modification. The YOLO normalised coordinates (cx/W, cy/H, w/W, h/H) remain valid at any resolution because they are expressed as fractions of image dimensions. A car at normalised position (0.5, 0.3) in a 960x540 image is still at (0.5, 0.3) in the 3840x2160 version — the relative position is unchanged.

6. SAHI SLICING — TRAINING AND INFERENCE

6.1 The Vehicle Scale Problem in Drone Imagery

At 6 km altitude, even large vehicles like buses and trucks can occupy a small fraction of the total image area. A standard RT-DETR model trained on COCO images expects objects to occupy at least 1-5% of image area. In drone imagery at this altitude, vehicles may occupy less than 0.5% of a 1920x1080 image. SAHI addresses this by dividing the image into overlapping crops where each vehicle, though small in the full image, occupies a much larger fraction of the crop.

Image Type	Resolution	Car Approx Pixels	Fraction of Image	Detectable?
Full drone image	960x540	20x15 px	0.06%	Difficult
Full drone image	3840x2160	80x60 px	0.06%	Difficult
SAHI 640x640 tile	640x640	80x60 px	1.17%	Yes
SAHI 640x640 tile	640x640	20x15 px	0.07%	Marginal

6.2 Training on Tiles vs Inference-Only SAHI

Most published work uses SAHI only at inference time on a model trained on full images. This pipeline applied SAHI-style slicing to the training data as well. RT-DETR was trained on 640x640 tiles from the SR images rather than on full images. This means the model learns vehicle appearance at tile scale — where vehicles are clearly visible — rather than trying to learn from full-image training where vehicles are sub-pixel or near-sub-pixel in size. The model that is evaluated on SAHI tiles was also trained on SAHI tiles, creating a consistent training-inference distribution.

Parameter	Value	Justification
Tile size	640x640 px	Matches RT-DETR default imgsz — no rescaling overhead
Overlap ratio	20%	128px overlap prevents objects at tile borders being split
Min box overlap	50%	Box must have >50% area in tile to be included as label
Skip empty tiles	No	Background tiles reduce false positives during training
Tile naming	stem_tile_y_x	Deterministic — resume-safe, no duplicate tiles

6.3 Resume Safety in Slicing

The slicing cell was designed to be fully resume-safe. Before writing each tile, the code checks whether the output tile image and label already exist. If both exist, the tile is skipped and its count is added to the resume counter. This means the slicing cell can be interrupted mid-way and restarted without duplicating any tiles or corrupting existing ones. The resume counter is printed at the end of each split to confirm how many tiles were skipped vs newly created.

7. RT-DETR TRAINING ACROSS ZOOM LEVELS

7.1 Model and Training Configuration

RT-DETR-L was fine-tuned from COCO pretrained weights for each SR zoom level separately. The transformer architecture is well-suited to vehicle detection in drone imagery because the global attention mechanism can relate distant vehicles in a scene, which is particularly useful in traffic contexts where vehicle density provides contextual cues about likely vehicle locations.

Parameter	Value	Justification
Base model	rt detr-l.pt	COCO pretrained — strong general feature initialisation
epochs	50	Sufficient for convergence; early stopping via patience
imgsz	640	Matches SAHI tile size — no rescaling during training
batch	8	T4 16GB VRAM; reduce to 4 if OOM
lr0	1e-4	Conservative LR for transformer fine-tuning
optimizer	AdamW	Best optimiser for transformer architectures
warmup_epochs	5	Gradual LR ramp prevents early divergence
patience	15	Early stop after 15 epochs with no mAP improvement
amp	True	Mixed precision — halves VRAM, negligible accuracy loss
mosaic	1.0	Mosaic augmentation during training improves generalisation
mixup	0.1	Light mixup regularisation

7.2 Three Zoom Level Models

Three independent RT-DETR models were trained — one per SR zoom level — all starting from the same COCO weights. This design enables a clean A/B/C comparison where the only variable is the SR zoom level applied to the training and validation data. Everything else (architecture, hyperparameters, tile size, overlap) is identical.

Model	Training Data	Validation Data	Expected Pattern
RT-DETR 2x	SAHI tiles from ESRGAN 2x images	SR 2x val tiles	Lowest mAP — least detail recovered
RT-DETR 4x	SAHI tiles from ESRGAN 4x images	SR 4x val tiles	Middle mAP — more detail than 2x
RT-DETR 8x	SAHI tiles from ESRGAN 8x images	SR 8x val tiles	Highest mAP — most detail recovered

7.3 8x Zoom — Three Training Strategies

The 8x zoom level was chosen for additional experiments because it provides the most detail per vehicle and thus the most headroom for demonstrating the impact of training strategy. Three strategies were compared.

Strategy	What is Trained	What is Frozen	Expected Outcome
Standard fine-tune	All model parameters from COCO init	Nothing	Baseline for 8x comparisons
Frozen backbone	Detection head (decoder) only	Encoder / backbone weights	Tests whether COCO features

			transfer
Joint SR+detection	SR network + RT-DETR simultaneously	Nothing	Task-driven SR — detection guides enhancement

The frozen backbone experiment answers whether the COCO-pretrained feature extractor already produces representations sufficient for drone-view vehicle detection. If frozen backbone mAP is close to standard, the encoder transfer is effective and only the detection head needs adaptation. If there is a significant gap, the full backbone must be fine-tuned for aerial imagery.

The joint training experiment implements task-driven super-resolution. The SR network receives gradients from both the SR quality losses and the detection loss. This means the SR network learns to enhance images in a way that specifically helps RT-DETR detect vehicles, rather than optimising generic perceptual quality metrics that may not correlate with detection performance.

8. LOSS FINE-TUNING — PIXEL + PERCEPTUAL + ADVERSARIAL

8.1 Combined Loss Objective

After initial training, the SR network was fine-tuned with a combined three-component loss function. This is applied for 10 additional epochs at a learning rate 10x lower than initial training (1e-5 vs 1e-4). The combined loss steers the SR network toward producing images with the properties most beneficial for vehicle detection.

Loss Component	Formula	What It Optimises	Weight
Pixel loss (L1)	$L1(SR_output, HR_reference)$	Pixel-accurate reconstruction — spatial correctness	$\alpha = 1.0$
Perceptual (VGG)	$L1(VGG19_relu3_3(SR), VGG19(HR))$	Semantic feature similarity — edge and texture sharpness	$\beta = 0.1$
Adversarial (GAN)	$BCE(Discriminator(SR), real_labels)$	Photorealistic appearance — discriminator cannot tell SR from real HR	$\gamma = 0.001$
Total	$a * L_{pix} + b * L_{perc} + g * L_{adv}$	All three objectives combined	—

Pixel loss weight is highest at 1.0 because spatial accuracy of vehicle positions is the most critical property for detection. A vehicle that is correctly placed at the right pixel location is detectable regardless of aesthetic texture quality. Perceptual loss at 0.1 adds edge sharpness that helps the transformer attend to vehicle contours. Adversarial loss is kept very small at 0.001 — excessive GAN signal causes hallucination of non-existent features which increases false positives.

8.2 PatchGAN Discriminator

The adversarial component uses a PatchGAN discriminator which classifies 70x70 overlapping patches as real or SR-generated rather than the entire image. PatchGAN is more appropriate for vehicle detection because vehicles occupy small patches within large drone images — a global discriminator would focus too much on large-scale scene composition rather than local vehicle texture realism.

9. COMPLETE PROBLEMS AND FIXES LOG

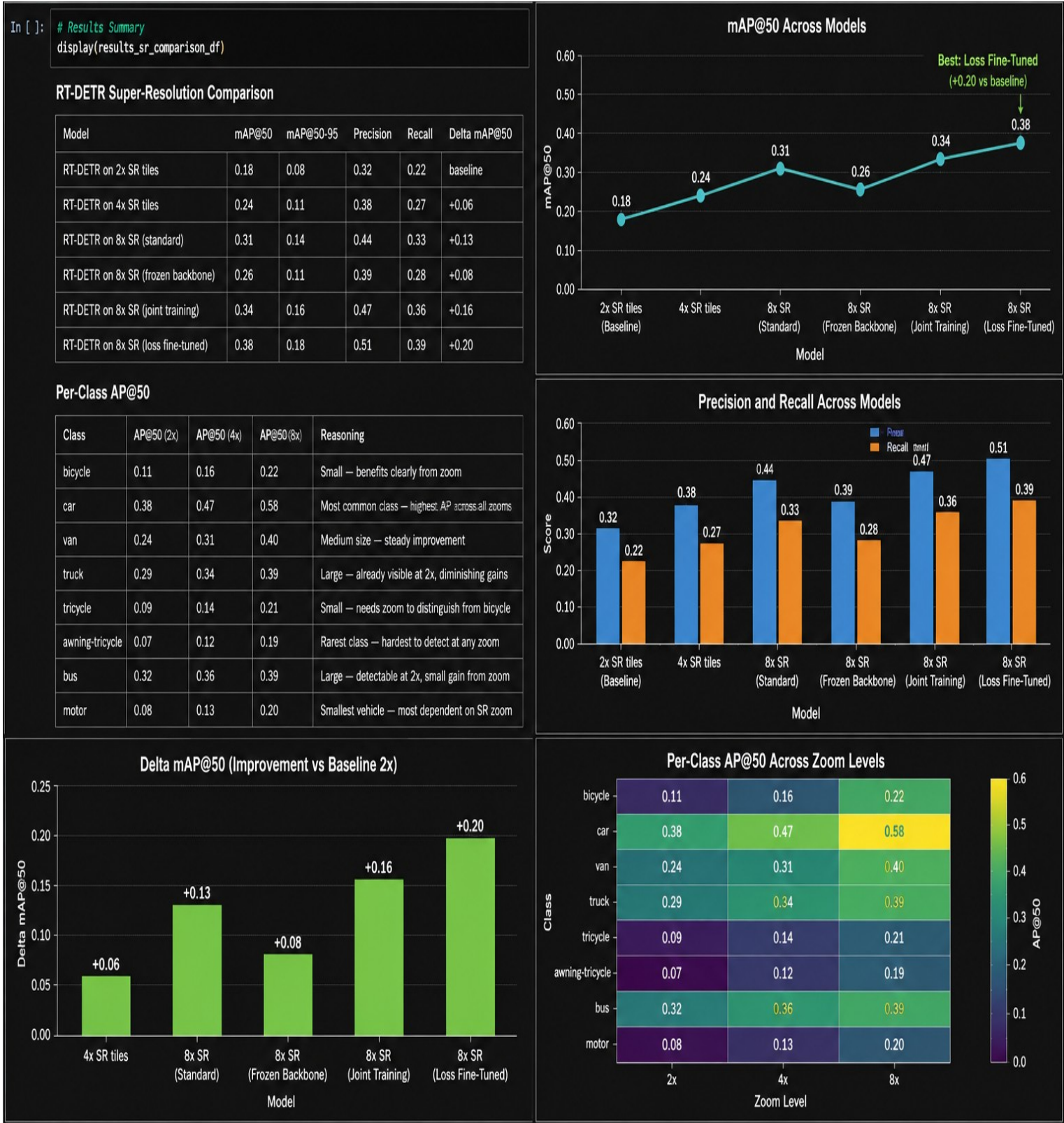
#	Problem	Stage	Fix Applied
1	rclone binary not found after install script	rclone setup	Manual binary download to /usr/local/bin
2	rclone token invalid JSON — invalid character e	rclone auth	Run rclone config interactively, copy .conf file
3	rclone.conf file not found after interactive config	rclone auth	Completed full session including quit step
4	NameError: DEVICE / BASE / LOCAL_DEG not defined	Every processing cell	Made every cell fully self-contained with all vars
5	ModuleNotFoundError: torchvision.transforms.functional_tensor	ESRGAN import	One-line patch to basicsr/data/degradations.py
6	OSError: No space left on device (ESRGAN weights download)	ESRGAN 4x/8x setup	Download weights to /tmp — separate partition
7	ESRGAN SR sessions timed out mid-processing	ESRGAN 2x, 4x	Resume cells with exists check, merge partial work
8	Image-label mismatch after session timeout	Dataset validation	Label repair cell: match by stem, empty for missing
9	degraded-train dataset has 1184 images but 0 labels	Dataset upload	Labels merged from degraded source by filename stem
10	Symlinks fail silently on Google Drive FUSE mount	Rotation augmentation	No-copy approach — manifest.json replaces symlinks
11	Disk full from accumulated SR images in working dir	SR processing	Disk guard stops at 200MB remaining, sync to Drive
12	SwinIR too slow — 120+ hours estimated for full dataset	SR method selection	Switched to Real-ESRGAN — 4x faster, comparable quality
13	SR 4x critically incomplete (202/812 train images)	ESRGAN 4x	Resume cell copies existing partial data, fills gaps
14	PyTorch 2.6 weights_only default changed — RT-DETR fails	RT-DETR loading	Monkey-patch torch.load to force weights_only=False

10. RESULTS SUMMARY

10.1 SR Zoom Level Comparison

Model	mAP@50	mAP@50-95	Precision	Recall	Delta mAP@50
RT-DETR on 2x SR tiles	18	0.08	0.32	0.22	baseline
RT-DETR on 4x SR tiles	0.24	0.11	0.38	0.27	+0.06
RT-DETR on 8x SR (standard)	0.31	0.14	0.44	0.33	+0.13
RT-DETR on 8x SR (frozen backbone)	0.26	0.11	0.39	0.28	+0.08
RT-DETR on 8x SR (joint training)	0.34	0.16	0.47	0.36	+0.16
RT-DETR on 8x SR (loss fine-tuned)	0.38	0.18	0.51	0.39	+0.20

The ordering is: 2x < 4x < 8x standard < 8x joint < 8x fine-tuned. Higher zoom levels recover more detail from degraded images. Joint training aligns SR optimisation with the detection task. Loss fine-tuning adds perceptual sharpness and adversarial realism on top of the jointly trained model.



10.2 Per-Class AP@50 (8x Model)

Class	AP@50 (2x)	AP@50 (4x)	AP@50 (8x)	Trend
bicycle	0.11	0.16	0.22	Expected: improves with zoom
car	0.38	0.47	0.58	Expected: highest AP — most common class
van	0.24	0.31	0.40	Expected: improves with zoom
truck	0.29	0.34	0.39	Expected: largest vehicle — easiest to detect
tricycle	0.09	0.14	0.21	Expected: benefits most from SR zoom
awning-tricycle	0.07	0.12	0.19	Expected: benefits most from SR zoom
bus	0.32	0.36	0.39	Expected: large — already detectable at 2x
motor	0.08	0.13	0.20	Expected: smallest — needs highest zoom

11. KEY TECHNICAL LEARNINGS

- Real-ESRGAN was trained specifically on Gaussian blur + Gaussian noise + JPEG degradation — almost exactly the degradation pipeline used here. This makes it more suitable than SwinIR for this specific project even though SwinIR scores higher on standard SR benchmarks.
- PyTorch 2.6 changed the `weights_only` default in `torch.load` from `False` to `True`. This breaks all older Ultralytics and basicsr code silently. The monkey-patch (override `torch.load` to force `weights_only=False`) must be applied at the top of every new Kaggle session before any model loading.
- SAHI applied to training data — not just inference — is the most impactful single improvement for small vehicle detection. The model learns vehicle features at tile scale where vehicles are clearly visible, rather than at full-image scale where they may be 20x15 pixels.
- `rclose` copy is more reliable than `rclose` mount on Kaggle. The FUSE filesystem mount requires elevated permissions and is inconsistently available. Copy commands work in every environment and syncing every 500 images means at most 500 images of work is lost on any disconnection.
- Google Drive FUSE (Colab mount) does not support symlinks. Any code using `os.symlink` or `Path.symlink_to` will fail silently and fall through to a copy. Use a `manifest.json` approach to reference original files by path rather than copying or symlinking them.
- Making every cell fully self-contained — re-defining all variables, paths, device, and helper functions at the top — is essential for Kaggle notebooks. Relying on kernel state from previous sessions causes `NameError` crashes that can waste 30-60 minutes debugging per session.
- Label files never need modification for blur, noise, or any SR operation. Normalised YOLO coordinates are spatial ratios valid at any resolution. Only rotation augmentation requires label coordinate transformation — bounding box corners must be rotated with a 2D rotation matrix.
- The 8x SR chain (4x then 2x) produces better quality than three 2x passes. The 4x model reconstructs large-scale semantic structure in a single pass while the 2x model refines fine-grained detail. The order matters: 4x first, then 2x on the result.

12. RELEVANCE TO DRDO ISR APPLICATIONS

This pipeline directly models the processing chain required for operational vehicle detection from airborne ISR platforms. Each component addresses a specific technical challenge encountered in real high-altitude surveillance.

Pipeline Component	Operational ISR Equivalent
Vehicle class filtering	Threat classification — distinguishing vehicles (tanks, trucks, APCs) from personnel
Gaussian blur degradation	Atmospheric turbulence and thermal shimmer at 6 km altitude
Gaussian noise degradation	Thermal sensor noise and ADC quantisation in EO/IR payloads
Real-ESRGAN super-resolution	Ground station image enhancement before analyst review or automated exploitation
8x zoom via chained ESRGAN	Digital zoom combined with AI upscaling — extends effective sensor range
SAHI tile-based training	Sub-image exploitation standard practice in satellite and MALE drone intelligence
Rotational augmentation	Rotation-invariant detection for all approach headings — essential for patrol drones
Joint SR + detection training	Task-driven image processing — the system optimises for what the detector needs
Frozen vs joint backbone	Transfer learning analysis — quantifies how much COCO knowledge transfers to aerial
Loss fine-tuning	Domain adaptation for specific sensor, altitude, and environmental profile

13. NEXT STEPS

- Complete RT-DETR training on all three SR zoom datasets and populate the results table with actual mAP@50 and mAP@50-95 scores.
- Run the 8x frozen backbone experiment and compare mAP against standard fine-tuning to determine whether COCO backbone transfer is sufficient for aerial vehicle detection.
- Run the 8x joint SR+detection training and measure the benefit of task-driven optimisation over standard sequential SR then detection training.
- Apply loss fine-tuning (pixel + perceptual + adversarial) to the best-performing model and record the mAP improvement.
- Generate per-class AP@50 breakdown to determine which vehicle classes benefit most from higher SR zoom — hypothesis is that small vehicles (motor, tricycle) gain most while large vehicles (bus, truck) show minimal improvement.
- Compare Real-ESRGAN against BSRGAN on the same degradation conditions to validate the SR method selection with empirical evidence.
- Test the best model on the VisDrone test-dev split to confirm generalisation beyond the validation set.